

LAPORAN AKHIR PRAKTIKUM

Mata Praktikum : Algoritma Pemrograman 2B
Kelas : IIA04
Praktikum ke- : 1
Tanggal : 14 April 2026
Materi : API
NPM : 50425612
Nama : Maulana Abdul Aziz
Ketua Asisten : Ni Gusti Ayu
Jumlah Lembar : 6



LABORATORIUM TEKNIK INFORMATIKA
UNIVERSITAS GUNADARMA

2026

1. Jelaskan secara singkat mengenai arsitektur API (REST, SOAP, RPC, dan sebagainya)!

API (*Application Programming Interface*) adalah antarmuka teknis yang berfungsi sebagai kontrak komunikasi antar sistem perangkat lunak terdistribusi, memungkinkan interoperabilitas dan pertukaran data secara terstruktur. Pemilihan arsitektur API tidak bersifat universal; keputusan ini ditentukan oleh parameter seperti kompleksitas data, kebutuhan keamanan, dan toleransi latensi. Berikut arsitektur API yang umum digunakan:

- **REST** (*Representational State Transfer*): Beroperasi di atas protokol HTTP dengan pendekatan berbasis sumber daya (*resource-based*). Sumber daya dikelola melalui URL endpoint menggunakan metode HTTP standar (GET, POST, PUT, DELETE) dengan format respons JSON. REST banyak digunakan pada sistem terdistribusi karena skalabilitas dan kompatibilitasnya yang tinggi.
- **SOAP** (*Simple Object Access Protocol*): Protokol berbasis XML dengan struktur pesan (*envelope*) yang ketat. SOAP menyediakan keamanan bawaan tingkat lanjut dan jaminan konsistensi transaksi (ACID), sehingga menjadi standar pada integrasi sistem enterprise yang kritis, seperti perbankan dan asuransi.
- **GraphQL**: Arsitektur yang memberikan kendali penuh kepada *client* untuk mendefinisikan struktur data yang dibutuhkan dalam satu *request*. Pendekatan ini mengeliminasi inefisiensi *over-fetching* yang umum terjadi pada REST, dan cocok untuk ekosistem dengan variasi kebutuhan data tinggi antar *client* (misalnya platform *mobile* vs. *web*).
- **RPC** (*Remote Procedure Call*): Berfokus pada eksekusi prosedur atau fungsi di server eksternal, seperti `getUser()`, seolah fungsi tersebut dijalankan secara lokal. Pola ini menghasilkan *payload* minimal, sehingga optimal untuk komunikasi internal antar-mikrolayanan (*microservices*) yang menuntut latensi rendah dan performa tinggi.
- **WebSocket**: Membangun saluran komunikasi dua arah (*full-duplex*) yang persisten antara *client* dan server. Berbeda dengan siklus *request-response* HTTP, WebSocket memungkinkan server melakukan *push* data secara proaktif tanpa permintaan eksplisit dari *client*, menjadikannya pilihan utama untuk aplikasi *real-time* seperti *live trading*, *chat*, dan *dashboard* monitoring.

Tabel Arsitektur API

Arsitektur	Protokol Utama	Format Data Dominan	Fokus Paradigma	Use Case Optimal
REST	HTTP	JSON, XML, dll.	Sumber Daya (<i>Resources</i>)	Layanan <i>web</i> publik, API <i>mobile</i> , CRUD standar.

SOAP	HTTP, SMTP	Eksklusif XML	Pesan (<i>Envelopes</i>)	Sistem finansial/perbankan, transaksi <i>enterprise</i> .
GraphQL	HTTP	JSON	Kebutuhan Data <i>Client</i>	Aplikasi <i>front-end</i> yang kompleks, agregasi API.
RPC	HTTP/2, TCP	Protobuf, JSON	Aksi/Fungsi (<i>Procedures</i>)	Mikroservis <i>backend-to-backend</i> (<i>low latency</i>).
WebSocket	WS, WSS	Teks, Biner	Koneksi Persisten (<i>Real-time</i>)	Aplikasi <i>chat</i> , <i>live streaming</i> , <i>dashboard real-time</i> .

2. Jelaskan logika program dalam activity secara detail!

```
api_maul.py > ...
1  import requests
2  payload = {
3      "Nama" : "Maulana Abdul Aziz",
4      "Kelas" : "1IA01",
5      "NPM" : "50425612"
6  }
7
8  parameter = {
9      'id' : 2
10 }
11
12 addpost = requests.post(
13     "https://jsonplaceholder.typicode.com/posts",
14     data=payload
15 )
16
17 getPostById = requests.get(
18     "https://jsonplaceholder.typicode.com/posts",
19     params=parameter
20 )
21
22 if (addpost.status_code == 201):
23     print("Data Berhasil Dikirim")
24     print(addpost.text)
25 else:
26     print("Data Gagal Dikirim")
27
28 print(getPostById.text)
29 print(getPostById.status_code)
```

Program `api_maul.py` ini merupakan sebuah implementasi praktis yang mendemonstrasikan interaksi fundamental dalam arsitektur REST API, khususnya operasi pengiriman data (*POST*) dan pengambilan data (*GET*). Narasi berikut mendekomposisi setiap blok kode untuk memahami mekanisme kerja serta rasionalitas teknis di baliknya.

1. Inisialisasi Library Requests

```
1  import requests
```

Baris perintah `import requests` berfungsi untuk mengintegrasikan library eksternal ke dalam lingkungan Python. Mengingat library ini bukan merupakan modul bawaan (*standard library*), tahap instalasi melalui `pip install requests` bersifat wajib. Penggunaan library ini secara signifikan mengabstraksi kompleksitas protokol HTTP, sehingga pengembang tidak perlu menangani detail teknis komunikasi soket secara manual.

2. Konstruksi Struktur Data (Payload)

```

2  payload = {
3      "Nama" : "Maulana Abdul Aziz",
4      "Kelas" : "1IA01",
5      "NPM" : "50425612"
6  }

```

Program mendefinisikan sebuah *dictionary* bernama `payload` yang memuat entitas data berupa Nama, Kelas, dan NPM . Dalam konteks RESTful API, struktur ini berperan sebagai *Request Body* yang akan ditransmisikan ke server. Secara teknis, format *dictionary* Python memiliki keselarasan struktural dengan JSON, yang merupakan standar industri untuk pertukaran data antara aplikasi .

3. Konfigurasi Parameter Query

```

8  parameter = {
9      'id' : 2

```

Selain *payload*, program mendefinisikan *dictionary* `parameter` yang berisi `'id': 2` . Berbeda dengan *payload* yang berada di dalam isi pesan, parameter ini dikonversi menjadi *query string* pada URL (misalnya: `.../posts?id=2`) . Mekanisme ini memberikan instruksi spesifik kepada server untuk melakukan filter data dan hanya mengembalikan entitas dengan identitas (ID) yang sesuai.

4. Eksekusi Permintaan POST

```

11  addpost = requests.post(
12      "https://jsonplaceholder.typicode.com/posts",
13      data=payload
14  )

```

Melalui fungsi `requests.post()`, program melakukan pengiriman data ke *endpoint* API . Target URL yang digunakan (`jsonplaceholder.typicode.com`) merupakan layanan *Fake API* yang mensimulasikan perilaku server nyata untuk kebutuhan pengembangan . Seluruh atribut respons dari server, termasuk kode status dan data kembali, dienapsulasi ke dalam variabel `addpost`.

5. Eksekusi Permintaan GET

```

16  getPostById = requests.get(
17      "https://jsonplaceholder.typicode.com/posts",
18      params=parameter
19  )

```

Setelah operasi POST, program menjalankan permintaan GET menggunakan `requests.get()` . Perbedaan krusial terletak pada penggunaan argumen `params=parameter`, yang menginstruksikan library untuk menyusun URL dengan *query string* secara otomatis, alih-alih mengirimkan data di dalam badan pesan (*body*) . Hasil dari operasi ini disimpan dalam variabel `getPostById`.

6. Verifikasi Status Operasi (POST)

```
21 if (addpost.status_code == 201):  
22     print("Data Berhasil Dikirim")  
23     print(addpost.text)  
24 else:  
25     print("Data Gagal Dikirim")
```

Untuk memastikan integritas transaksi, program melakukan validasi terhadap `addpost.status_code`. Kode status merupakan indikator numerik dari server; dalam hal ini, nilai `201` (Created) menjadi konfirmasi bahwa data baru telah berhasil diakomodasi oleh server. Jika kondisi terpenuhi, sistem akan mencetak konfirmasi keberhasilan beserta teks responsnya.

7. Output dan Validasi Data (GET)

```
27 print(getPostById.text)  
28 print(getPostById.status_code)
```

Tahap akhir program melibatkan visualisasi hasil dari operasi GET. Perintah `print(getPostById.text)` akan menampilkan data dalam format JSON yang ditarik dari server, sementara `status_code` yang bernilai `200` (OK) memberikan verifikasi final bahwa permintaan pengambilan data telah diproses dengan sempurna oleh server.

Kesimpulan Alur Kerja

Secara holistik, program ini mencerminkan siklus hidup penuh dari konsumsi API: mulai dari persiapan data, transmisi menggunakan metode HTTP yang tepat, hingga penanganan respons server. Pola sistematis ini merupakan fondasi utama dalam pengembangan aplikasi modern yang berbasis layanan mikro atau integrasi sistem pihak ketiga.